

```
$ cat notes.md
```

DevOps & Cloud [SWE] — Class 9 Notes

LECTURE TITLE Docker Compose & Kubernetes Fundamentals

DATE/TIME 1 September 2026, 11:00 AM

DURATION 120 minutes

INSTRUCTOR Nensi Ravaliya

BIG IDEA Docker Compose runs a handful of containers on one machine. Kubernetes runs millions of them across a fleet — and the whole architecture exists to answer one question: *where should this container go, and is it still alive?*

SOURCES this class's transcript + `Class-09/compose.yaml` (yours) + the instructor's repo [Nency-Ravaliya/devops-heros](#) → `session8-docker-networking-volume/demo/` and `session9-k8s/`

WHAT'S INSIDE — 20 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 🗉 Recap Round — Networks & Volumes
- 02 📁 The New Submission System
- 03 🗉 Dockerfile Instructions — Rapid-Fire Recap
- 04 🐳 Docker Compose — Why It Exists
- 05 ⚡ YAML — A 60-Second Primer
- 06 📁 Anatomy of a Compose File
- 07 📋 Compose Commands
- 08 🖥️ The Full 3-Tier Demo
- 09 💬 The Class Debate — Is Docker Compose Actually Used?
- 10 🗉 Docker — Full Course Recap
- 11 🌐 Why Kubernetes Exists
- 12 🌐 Kubernetes Architecture
- 13 💠 The Control Plane — Four Components
- 14 🖥️ The Worker Node — Two Components + a Runtime
- 15 📦 Pod, Node, Container — The Nesting
- 16 💠 Career Asides
- 17 📝 Homework
- 18 📄 Cheat-Sheet
- 19 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. theme: `Compose Teal`

00



The 60-Second Version

1. **Docker Compose = one YAML file that starts your whole app.** Three containers, two networks, a volume — `docker compose up -d` . Done.
2. **Multi-stage ≠ multi-container.** Multi-stage makes **one** image from one Dockerfile. Compose runs **many** containers.
3. **A Compose file has exactly three top-level blocks:** `services:` , `networks:` , `volumes:` .
4. `image:` pulls someone's image. `build:` builds yours from a folder containing a Dockerfile.
5. **Never publish a port for your backend or database.** Only the frontend gets `ports:` .
6. **Kubernetes has two halves:** the **control plane** (the boss — decides things, runs nothing) and **worker nodes** (where your app actually runs).
7. **Control plane = 4 components:** **etcd** (the database) · **API server** (the front door) · **scheduler** (where does this pod go?) · **controller manager** (is reality matching the plan?).
8. **Worker node = 2 components + a runtime:** **kubelet** (keeps containers alive) · **kube-proxy** (networking) · **containerd**.
9. **Cluster → Node → Pod → Container.** A **pod** is the smallest unit in Kubernetes.



01



Recap Round — Networks & Volumes

She opened by re-asking everything from Class 8.

Question	Answer
Does the none network have a driver?	No — <i>"that's why we call it a none network"</i>
Can a none-network container reach Google?	✗ No
What does host network do?	shares your local machine's network with the container
Why does bridge need <code>-p 8080:80</code> ?	because bridge is private/internal — you must forward the port
Host network command?	<code>docker run -dit --name web --network host nginx</code> — no -p
How many volume types?	two — default Docker volume and bind mount
When do you use a bind mount?	<i>"when we want to bind my current folder, or any folder of my local machine, to a container"</i>
Flag for a volume?	<code>-v <volume-name>:<path></code>

Question	Answer
Create a volume?	<code>docker volume create <name></code>
Can two containers in different user-defined networks talk?	✗ No — and that's the point

NOTE

On the host-network Mac issue from last class: *"30% of people are getting the nginx webpage, and for Mac users they are not."* (See Class 8 §10 — it's Docker Desktop's Linux VM, not your browser.)



02 The New Submission System

She rolled out a proper structure for homework.

The folder layout

```

devops-heros/           ← your fork
├── session1-.../
├── session8-.../
│   ├── docker-compose-app/
│   ├── nginx-web/
│   ├── node-app/
│   └── README.md        ← ONE readme per session
└── session9-.../

```

What goes in the README

LOCK THIS IN

Per task: *"just add one line description — this is the task — then the screenshot of that output. Then the second task, and the screenshot of that output."*

How to submit

1. Pull the session folder she adds to the repo
2. Do the homework inside your own copy
3. Write ONE readme.md per session, with task + screenshot pairs
4. Copy the readme.md URL
5. Paste it into the Google Form (separate links for Section A and Section B)

💡 GOOD TO KNOW

"It's editable — if you added a wrong readme file, you can edit your response."

😊 CLASS MOMENT

Where the task list came from: a student compiled all the assignments into a document ("he added the transcript, I guess") and she modified it. "So four to five tasks we added, as per every session that we had."



03



Dockerfile Instructions — Rapid-Fire Recap

Before Compose, she quizzed the whole instruction set again.

Instruction	Answer given
FROM	base image
RUN	run a command
CMD	the triggering command for your app, at the end of the Dockerfile
EXPOSE	⚠️ see below
COPY	source → destination, local only
ADD	COPY + external zip / RPM / links
ENTRYPOINT	the main program — cannot be overridden (CMD can)

🎯 EXPOSE — and the answer to "why does it even exist?"

🎯 INTERVIEW ALERT

"`EXPOSE 80` in `nginx` — does that mean it's exported on port 80 in my browser?" → **No.** "It's just telling us your application is listening on port 80."

✅ **But then she gave the reason it's genuinely useful, which nobody usually explains:**

"Every time, developers are working with their codebases. We are not developers — but we need to know. They want to deploy a website, we are building a CI/CD pipeline. At that time we need to know that this backend component is running on port 5000. And how do I know it's running on 5000? That's why, for me — for the DevOps engineer — they add `EXPOSE 5000` inside the `Dockerfile`. So I get to know I need to redirect traffic from 5000 to some other port in my browser."

🧠 **EXPOSE is a message from the developer to you.** It's a contract, not a switch.

🎯 Multi-stage vs multi-container, one more time

⚠️ WATCH OUT

"Multi-container, or Docker Compose, and multi-stage file is a bit confusing. Sometimes after five or six months, if you're not using Docker, it's confusing."

	Multi-stage Dockerfile	Docker Compose
What it is	multiple stages inside one Dockerfile	multiple containers
How many containers at the end?	★ ONE	many
The two keywords	<code>AS builder</code> and <code>--from=builder</code>	<code>services:</code>

✅ LOCK THIS IN

"Make sure — from a multi-stage `Dockerfile` we're creating multiple stages inside a `Dockerfile`, but at the end it is going to create only one container."



04 🐳 Docker Compose — Why It Exists

🎯 The problem, counted out in steps

🎯 INTERVIEW ALERT

"Let's say I want to bring up my whole application — frontend, backend, DB — inside networks, and make sure the frontend is not able to connect to the database directly. Plus I need a volume for my database to save user data. How do I bring up everything at once?"

Without Compose, for a three-container app, that's:

1. create the frontend network
2. create the backend network
3. create the volume
4. run the frontend container (`--network frontend_net, -p`)
5. run the backend container (`--network frontend_net`)
6. connect backend to the DB network too
7. run the database container (`--network backend_net, -v`)
8. ...and repeat all of it every time you restart

✅ LOCK THIS IN

"But what if I tell you we can do everything at once by just one file? I can manage each container — up, down, scale down, everything — by just one file. That is what Docker Compose means."

Everything above becomes:

```
docker compose up -d
```



05 ⚡ YAML — A 60-Second Primer

🗣️ SAID IN CLASS

"How many of you know YAML? There are a lot of websites, games are there, so you can just play with those games to learn YAML."

✓ LOCK THIS IN

Her whole explanation, and it's honestly enough:

"YAML is almost like Python. If you know Python, YAML is very easy. Indentation is there, array is there and dictionary is there. Nothing else. Key-value, arrays, dictionaries. That's it."



```
key: value                # key-value

parent:                   # dictionary (nested by INDENTATION)
  child: value

list:                     # array
- item one
- item two
```

⚠ WATCH OUT

The one thing that will bite you: indentation is the syntax. YAML has no braces. Two spaces in the wrong place changes the meaning. And use **spaces**, **never tabs**.

The filename

Filename	Verdict
<code>docker-compose.yml</code>	✓ the standard <i>"most applications in 2024–2026 make sure the file name is this"</i>
<code>compose.yml</code>	✓ also works — the newer official name
<code>compost.yml</code>	<i>"some old formats / old applications"</i>



06 Anatomy of a Compose File

The three top-level blocks

```
services:      # ← your containers
  ...

networks:      # ← must be OUTSIDE services
  ...

volumes:       # ← must be OUTSIDE services
  ...
```

⚠ WATCH OUT

The most common structural mistake: "Make sure network and volume you need to specify OUTSIDE of the services. If you're using a network, you need to declare the network. If you're using a volume, you need to declare the volume — the same name you're using."

📁 Your own file — `Class-09/compose.yaml`

This is the simple version built in class:

```
services:
  frontend:
    image: nginx
    ports:
      - 8080:80

  backend:
    image: nginx

  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
    volumes:
      - db-data:/var/lib/mysql

volumes:
  db-data:
```

✓ LOCK THIS IN

Correct, and it demonstrates all four key ideas: `image:` for pulled images · `ports:` **only** on the frontend · `environment:` for the DB credentials · a **named volume** declared at the bottom and mounted in the service.

Line by line

services: — each entry is a container

✓ LOCK THIS IN

A naming note: "Frontend is the name of the container — just like `--name` when running a container. But we don't call it a container name, we call it a **SERVICE**."

image: vs **build:** — two ways to get an image

```
# Option 1 - pull a ready-made image
frontend:
  image: nginx:latest

# Option 2 - build from YOUR code
backend:
  build: ./backend      # ← folder containing a Dockerfile
```

✓ LOCK THIS IN

"If you have your own Dockerfile and you need to build an image out of it, then specify `build` instead of `image`, and then the location of the Dockerfile. So it will automatically go to the backend folder, search for the default Dockerfile, and build an image out of it."

ports: — and the rule about who gets one

```
ports:
  - "8080:80"
#   |      container port
#   |      HOST port
```

⚠ WATCH OUT

The rule she was emphatic about:

```
frontend → ✅ ALWAYS specify a port
backend  → ❌ NEVER
database → ❌ NEVER
```

Why: "Do I need to see my backend directly in my web browser? We never see. If we're doing this, that means we're exposing any kind of database, username, to the internet."

🔊 "We never expose our backend APIs to external traffic. Even if a developer is developing and is able to hit an API — that's their company laptop, their company's private network. Only the organisation's people are able to access those backend APIs, not any outside person."

environment: — variables for the container

```
environment:
  MYSQL_ROOT_PASSWORD: root
  MYSQL_DATABASE: demo
```

⚠ WATCH OUT

Her own caveat: "We never specify a password like this in actual production." (Real answer: Docker/Kubernetes **secrets**.) It's fine for learning.

volumes: — inside a service, and again at the bottom

```
database:
  volumes:
    - db_data:/var/lib/mysql    # ← same -v syntax as docker run

volumes:                        # ← and declare it at the top level
  db_data:
```

💡 GOOD TO KNOW

`/var/lib/mysql` is MySQL's **default data directory**. That's the path you always mount for MySQL.

networks: — the isolation layer

```
frontend:
  networks: [frontend_net]

backend:
  networks: [frontend_net, backend_net]    # ← in BOTH

database:
  networks: [backend_net]

networks:
  frontend_net:
  backend_net:
```

✓ LOCK THIS IN

That's Class 8 §9, expressed declaratively. The frontend and database share no network, so they cannot reach each other.

NEW depends_on: — startup ordering

```
backend:
  depends_on:
    - database
```

🔔 INTERVIEW ALERT

She asked the class and someone got it immediately: *"First — correct, absolutely correct."*

✓ *"When I do `docker compose up -d`, it will create the frontend, pull the nginx image. Then it sees the backend and searches for the keyword `depends_on`. If it's there, it will not create the backend first — it will check whether my database is up and running. If it is, then and only then will it move forward with creating the backend."*

🔍 **One useful precision:** `depends_on` waits for the container to have **started**, not for the database to be **ready to accept connections**. MySQL takes several seconds to initialise after its container starts, so a backend can still fail its first connection attempt. That's why her demo app's `app.py` imports `time` — real apps add a retry loop or a **healthcheck** on top of `depends_on`. Her explanation is the right mental model; just know there's a gap between "started" and "ready".



07 🎵 Compose Commands

```
docker compose up          # start everything, logs in the foreground
docker compose up -d       # ★ start in the background
docker compose up -d --build # rebuild images first, then start
docker compose ps          # list THIS project's containers
docker compose down        # stop and remove everything
```

Command	Does
<code>up</code>	build/pull + create networks, volumes, containers
<code>-d</code>	detach — "same as with containers"
<code>--build</code>	"it's actually building your Docker image layers again from the Dockerfile" — use it after you change your code
<code>ps</code>	"just like <code>docker ps</code> , but for Compose"
<code>down</code>	⚠️ stop — not <code>docker compose stop</code>

⚠️ WATCH OUT

A lesson she learned live, and it's a real gotcha:

```
cd ~/anywhere-else
docker compose ps      # ❌ shows nothing

cd ~/repo/session8/demo
docker compose ps      # ✅ shows your containers
```

✅ "Inside the folder you can only use Docker Compose. If I run the same command outside my folder, I'm not able to see even the containers. But for Docker commands — `docker ps` , `docker images` — it will run in any terminal. For Docker Compose we cannot do the same thing."

🧠 **Why:** Compose scopes everything to a *project*, and the project is inferred from the directory containing the compose file.

😬 Live-demo chaos

Three things went wrong, and each teaches something:

What happened	The lesson
Ran <code>up</code> without <code>-d</code> → all three containers' logs streamed by	that's what <code>-d</code> prevents
<code>Ctrl+C</code> → everything stopped	foreground Compose dies with the terminal
The frontend container wouldn't appear — <i>"port 8080 is already in use"</i> from an earlier nginx	⚠️ <code>docker compose down</code> before you <code>up</code> again, and check <code>docker ps</code> for stragglers
Ran <code>docker compose ps</code> in the wrong directory → empty	the directory rule above

😊 CLASS MOMENT

Plus a genuine mic failure that took several minutes: *"I need to call these people for mic issues... Every instructor is out of the mic."* 😊



08 🖥️ The Full 3-Tier Demo

This is the app in `session8-docker-networking-volume/demo/`, and it ties together everything from Classes 6–9.

```
demo/
├── docker-compose.yml
├── frontend/
│   ├── index.html    ← a page with a button
│   └── nginx.conf    ← reverse proxy config
└── backend/
    ├── Dockerfile
    ├── app.py        ← Flask + MySQL
    └── requirements.txt
```

The compose file

```
services:

  frontend:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:                                # ← BIND MOUNTS
      - ./frontend/index.html:/usr/share/nginx/html/index.html
      - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf
    networks:
      - frontend_net

  backend:
    build: ./backend                        # ← builds YOUR Dockerfile
    networks:
      - frontend_net
      - backend_net                         # ← in BOTH
    depends_on:
      - database

  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: demo
    volumes:
      - db_data:/var/lib/mysql              # ← NAMED VOLUME
    networks:
      - backend_net

networks:
  frontend_net:
  backend_net:

volumes:
  db_data:
```

✓ LOCK THIS IN

The two `volumes:` lines on the frontend are bind mounts from Class 8, doing exactly what she demonstrated then: *"I don't want to see their default page, I want to see my page. So I just need to replace their default file with my `index.html`."* Same for `nginx.conf` replacing `default.conf`.

The backend's Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

✓ LOCK THIS IN

Every instruction from Class 6 and 7, in six lines. And `EXPOSE 5000` is doing exactly the job she described in §3 — telling you the backend listens on 5000.

The nginx reverse proxy — the bit that makes it work

```
location /api {
    proxy_pass http://backend:5000/api;
}
```

✓ LOCK THIS IN

This is the answer to the whole "how do we not expose the backend?" question.

```
Browser → localhost:8080/api      (the ONLY published port)
         ↓ nginx sees /api
         → http://backend:5000/api (inside the private network)
```

The browser **never** touches port 5000. It talks only to nginx, and nginx forwards **one specific path** onwards. That's a **reverse proxy**, and it's how every real app does this.

🧠 And notice `http://backend:5000` uses the **container name as a hostname** — the user-defined-bridge DNS from Class 8 §5, doing real work.

🔪 The proof, run live

She exec'd into the **frontend** container and tried to reach the backend:

```
docker exec -it <frontend-container> /bin/bash

# ping doesn't exist in nginx – but curl does
curl http://backend:5000
# → Hello from Backend! ✓

curl http://backend:5000/api
# → {"backend": "Backend is working!", "database": "Hello from MySQL!"} ✓
```

And from your browser:

```
localhost:5000      ✗ nothing – the port was never published
localhost:8080/api  ✓ works – through the nginx proxy
```

✓ LOCK THIS IN

That's the entire security model demonstrated in two commands. The backend is reachable *inside* the network and unreachable *outside* it.

💡 GOOD TO KNOW

Her production aside: "In real use cases we're not doing 'hello from backend'. We have a health check. A specific path — `backend:5000/api/healthcheck` — and it gives you a string: your DB is up and running. In actual microservices you're going to implement that."



09



The Class Debate — Is Docker Compose Actually Used?

This was the liveliest exchange of the course, and it's worth recording honestly because both sides had a point.

Her position

SAID IN CLASS

"We are not using Docker Compose in the current industry. It's for learning purposes. We have Kubernetes, we have EKS. Why should I do all the containers, then scale up, scale down, when I have an EKS service?"

But then the important part: "In each interview that I had given — almost 100, maybe 150+ interviews (yeah, I failed almost all of them, and I learned from those) — the main thing they're going to ask you: write a Docker Compose file, write a Dockerfile, run this container, do this operation inside a container. That's why I added this topic. It's actually necessary and important."

✅ "Even they also know we're not using Docker Compose in industry — we're going to use the Kubernetes cluster. But before learning Kubernetes, you need to make sure you know the fundamentals."

The student's pushback

A student who runs their own company disagreed, and made a genuinely good argument:

💡 "The decision to use Kubernetes or not depends on the scale. You have a startup with 47 users — and you run a Kubernetes cluster?"

Her: "Yeah, that is one good point, yes. If you have only one or two users and you're running a Kubernetes cluster, we call it a duffer." 😊

Student: "Even at 10,000 users, or 100,000, or even a million — running Kubernetes depends very much on the team size and the cost the company is ready to accept. Why wouldn't I just run ECS?"

Her: "Yeah, you can use ECS. When I started my learning I went EC2 → ECS → ECR, and only rarely EKS, because it costs too much for learning."

The second debate: exposing backend ports

💡 **Student:** "My diagram looks like — there's a public subnet and a private subnet. Backend is in the private subnet. I will never expose the port of my backend from my private subnet to the internet."

Her: "Yeah. Yeah. That's what I'm saying."

✓ LOCK THIS IN

Where they actually landed, and it's the right synthesis:

For learning, hackathons, single small projects	Compose is fine, and exposing a port to test an API is fine
For an organisation at scale	never publish backend ports; orchestrate with Kubernetes/ECS
For interviews	you <i>will</i> be asked to write a Compose file

🎤 Her closing note on it: *"Sometimes we need to test an API — at that time we need to expose the port for the backend to test the APIs. But in a real environment we never do it. Even after testing, we never do."*

💡 GOOD TO KNOW

The fair reading: she's describing large-enterprise practice (she works at a bank, at scale, where Kubernetes is the default). The student is describing startup practice, where Compose or ECS on a single box is often the *correct* engineering choice. **Both are true at their own scale** — and knowing *which* scale you're at is itself the senior skill.



10 🔄 Docker — Full Course Recap

🎤 SAID IN CLASS

"So far, what we learned in Docker:"

- ✓ **Docker architecture** — client, daemon, host, registry
- ✓ **Container lifecycle** — created → running → paused → stopped → removed
- ✓ **Basic commands** — run, ps, exec, logs, stop, rm
- ✓ **System prune commands** — cleaning up dangling images and unused containers
- ✓ **Images & containers** — blueprint vs running instance
- ✓ **Volumes** — Docker volume vs bind mount
- ✓ **Networks** — bridge, host, none, overlay
- ✓ **Container-to-container communication**
- ✓ **DNS resolution via user-defined bridge networks**
- ✓ **Isolating the frontend from the database**
- ✓ **Multi-stage Dockerfile** (one container) **vs multi-container Compose** (many)

✓ LOCK THIS IN

Her framing for the transition: "If you're able to understand Docker and troubleshoot all the container-specific things, then Kubernetes will be very easy. At the base of Kubernetes we have containers. Kubernetes is nothing but a manager, managing all these containers."



11 Why Kubernetes Exists

The scaling ladder

🧠 PICTURE THIS

Picture the progression:

```
1 container           → docker run
a few containers      → Docker Compose
10–20 containers     → Docker Swarm
MILLIONS of containers → Kubernetes
```

🔧 "What if I have millions, or tens of containers, and I have to manage all of them? I need to scale down maybe 300 containers for 40+ services. I need to scale up some containers for database or backend. Or some team is working on a specific service, they updated a few things, and they need to roll back their existing infra and bring up a new updated version. Because of these types of requirements, we need Kubernetes."

🔔 What killed Docker Swarm

✓ LOCK THIS IN

Docker Swarm was Docker's own clustering tool, and it came first.

Swarm's problem

Scaling	"scaling is one of the problems"
Rollback	"rollback inside a Swarm cluster"
Complex deployments	"legacy applications — at that point the Swarm cluster will not work as expected"
Only suited to	"small applications, not complex and large applications"

💡 GOOD TO KNOW

But she was fair to it: *"Compared to Kubernetes, Swarm is very easy — easy to handle, easy to manage, easy to learn. But Kubernetes is a bit hard, because for each component we have a server, and for each component we have a quorum."*

✅ LOCK THIS IN

"Docker Swarm failed. That's the main thing. So Google developers were at work and thought — okay, let's create something for everyone."

And then the cloud providers wrapped it:

Cloud	Service
AWS	EKS — Elastic Kubernetes Service
Azure	AKS
GCP	GKE

😊 CLASS MOMENT

"For Google? Don't tell me GKE. ...GKE." 😊

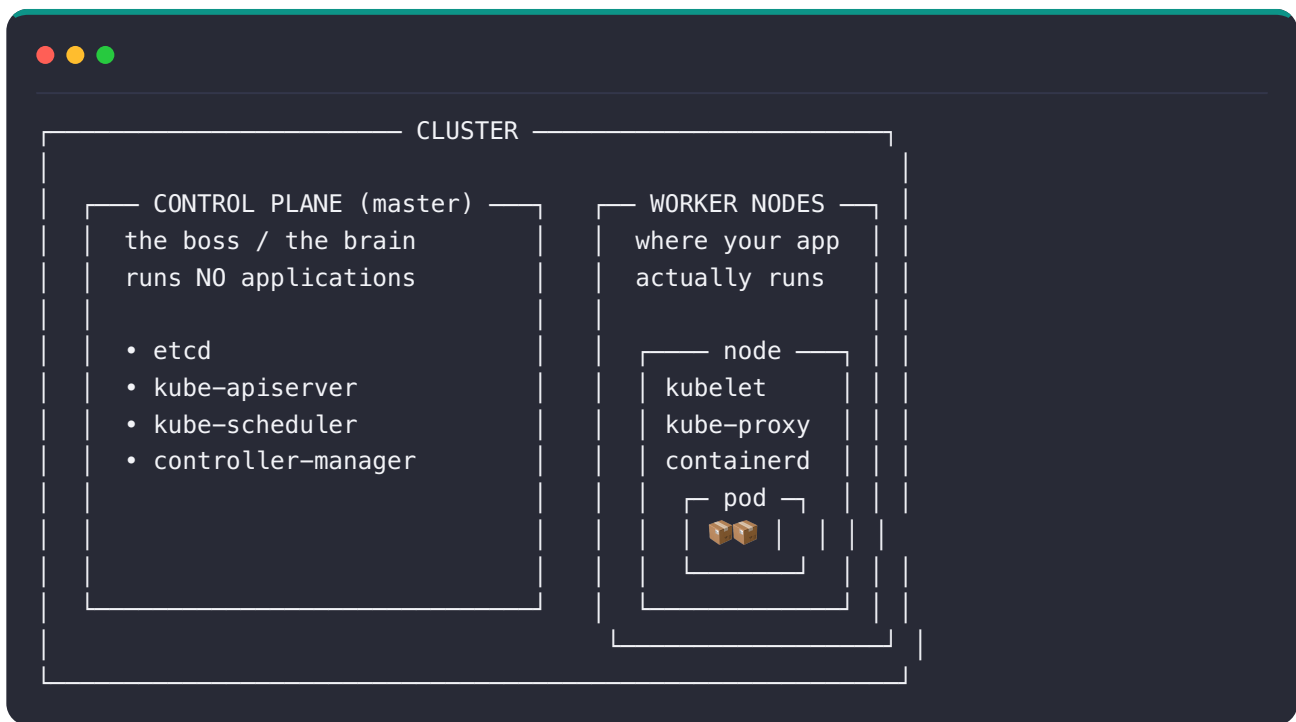
And the K8s trivia one more time: *"Kubernetes — k8s. I guess I told you that funny fact, right? 8 letters are there, that's why we call it k8s."*



12 🌀 Kubernetes Architecture

✅ LOCK THIS IN

The structural parallel she drew: *"Same way as we learned Docker. Docker architecture is client–server. Here we have Kubernetes architecture, where we have two things, only two components."*



🧠 PICTURE THIS

Her analogies for the two halves: "One is a boss, another are employees. One is a teacher, another are students."

⚠️ WATCH OUT

The most important structural fact: "Control plane / master node does NOT run any kind of application. First thing — it's just managing all the workloads."

🎯 The interview question she flagged for next class

🎯 INTERVIEW ALERT

"The control plane is the brain of Kubernetes. If the control plane crashes, then... your application may still be up and running, but you're not able to see all the functionality. This is an interview question — we'll cover it in the next lecture."

🧠 **Why:** existing pods keep running (the worker nodes don't need permission to keep doing what they're doing) — but nothing new can be scheduled, nothing can scale, nothing self-heals, and `kubectl` stops working. **The plane keeps flying; you just lost the cockpit.**

What a cluster actually is

LOCK THIS IN

"Whatever we have inside — control plane and worker node, boss and servant — they're both inside one environment. That environment we call a cluster. When we're creating an EKS service, we're creating a cluster environment. Inside that environment we have the master node and the worker nodes — that is what a cluster is."

Multiple masters and quorum

INTERVIEW ALERT

"Can we create multiple masters? That's what we have inside a quorum. If one master node fails, then who will handle all the workload?"

 "In industry, there are two to three master nodes. One cluster I'm working on has three master nodes. If one fails, another takes the workload."

 **The scale she's working at:** "it will check almost 7,200 services inside the worker nodes we have."

 **Quorum** is why the number is odd (3, 5, 7): etcd needs a **majority** to agree on the cluster's state. With 3 masters you can lose 1 and still have 2 of 3 — a majority. With 2 masters, losing 1 leaves you with no majority at all.



13 The Control Plane — Four Components

CLASS MOMENT

"See — cloud controller manager? Forget about it. Never existed."  (It's only relevant when your cluster is managed by a cloud provider.)

1 etcd — the database

What it is	a key-value storage database
What it stores	the state of Kubernetes — "each and every detail regarding your running containers"
Scale	"If 10,000 containers are running, each container's details are inside etcd"

LOCK THIS IN

"You can say etcd is the main storage of the complete cluster. Whatever is happening inside the cluster, everything will be there in etcd."

💡 GOOD TO KNOW

"Key-value database" — what that means, and what's actually in there.

No tables, no columns, no SQL. Just **a path, and the thing stored at that path** — like a giant filing cabinet, or a Python dictionary the size of your cluster:

KEY	VALUE
/registry/pods/default/nginx-abc123	→ { image: nginx:1.25, node: worker-2, status: Running, ip: 10.244.1.7 ... }
/registry/deployments/default/myapp	→ { replicas: 3, image: nginx:1.25 ... }
/registry/services/default/myapp-service	→ { type: NodePort, port: 80 ... }

✅ **Why a key-value store and not a normal database:** Kubernetes only ever needs "give me the object at this exact path" and "tell me when anything under this path changes." It never needs to join or aggregate. A key-value store does those two things extremely fast, and can be replicated across three machines for safety (which is what **quorum** is for).

🔗 **And here's the fact that makes the whole architecture click:** `kubectl get pods` doesn't ask the worker nodes anything. **It reads etcd.** Every component in Kubernetes is really just reading and writing this one database through the API server — that's the entire design.

2 kube-apiserver — the front door 🚪

🧠 PICTURE THIS

Her analogy, and it's perfect: "This is the front door of your Kubernetes cluster. If you want to enter the cluster, you need to speak with the API server. If the API server allows you, then and only then are you able to deploy any kind of application."

"It's like a door. We have two doors here — but what if we had only one? If you want to enter this classroom, you have to come to that door, open it, and then you can enter."

💡 And the deeper insight she called out:

"Whenever you're writing deployment files, YAML files for Kubernetes, you're writing `apiVersion` and `kind`. API version means everything inside Kubernetes is an API object. One of the good learnings — after 20 years working on Kubernetes, people don't know this. Everything you create in Kubernetes is an API object."

✅ **That's why `kubectl` talks to an API server:** a Pod, a Deployment, a Service, a Secret — every one of them is a REST object you create, read, update and delete. Kubernetes is a database with a scheduler bolted on.

3 kube-scheduler — where does this pod go? 📌

✅ LOCK THIS IN

"Scheduler's work is to get information from the API server, and schedule a particular pod inside a node."

How it decides:

```
API server → asks etcd: "what does this service need?"
                → "8 cores of CPU, 10 GB memory"
API server → tells scheduler: "here's the service, here's the requirement,
                                can you schedule it on a suitable worker node?"
Scheduler → checks node 1: "do you have this much CPU and memory free?"
                → node 1 already runs 100 services. No.
                → checks node 2: → "yes"
                → ✅ "this service will deploy on node 2"
```

🧠 PICTURE THIS

The scheduler is a matchmaker. It never runs anything. It just decides *where*.

4 controller-manager — is reality matching the plan? 🖥️

✅ LOCK THIS IN

The core idea: the controller constantly compares **what you asked for** against **what actually exists**, and fixes the difference.

Controller	Watches
Node controller	<i>"Node is up and running? I need four nodes — are there four?"</i>
ReplicaSet controller	<i>"In node 1 I need only 5 pods — are there exactly 5?"</i>
others	deployments, endpoints, and more

✅ LOCK THIS IN

"Controller is the component managing your worker nodes — application deployment, node information, which port, taints and tolerations. This is a bit advanced, but when you get to the commands you'll understand what the controller is actually doing."





1 kubelet — keeps containers alive 💖

✓ LOCK THIS IN

"Kubelet's task is simple. If I say I want three containers for frontend, it makes sure three containers will be there. If one is down, it creates it again. If two are down, it creates them. If there are four, it removes the fourth. Its only task is to check your container is up and running."

And the heartbeat:



```
kubelet → every few seconds: "is the container up?"  
        → yes → 💖 sends a HEARTBEAT to the API server  
API server → tells the controller: "these are up and running"  
controller → "I need only three frontends"  
API server → kubelet: "make it three"  
kubelet    → 2 running? create 1.  4 running? remove 1.
```

🔍 CLARIFIED

One small precision on that loop: in the real flow, the **kubelet** reports node and pod status **up** to the API server, and the **controller manager** watches the API server and writes the desired state back down. The kubelet then acts on what it reads for its own node. Her description has all the right pieces and the right feedback loop — just note that the components don't call each other directly; **everything goes through the API server**. That's why it's called the front door.

2 kube-proxy — networking 🕸

✓ LOCK THIS IN

"It's basically the network policies we're enforcing on my pods. Anything you want to add, delete or update in networking, you're going to use kube-proxy. Kube-proxy is the manager in the network department."

💡 GOOD TO KNOW

It's the Kubernetes equivalent of Class 8's Docker networks.

3 Container runtime (CRI) 📦

🎯 INTERVIEW ALERT

The bit of history worth knowing:

"We are not basically using the Docker daemon, or actual Docker, as the runtime in Kubernetes. We are using containerd. Previously, when Kubernetes first came, we were using Docker inside each pod as the container runtime interface. But after that, Docker was not supporting enough in Kubernetes. So they made some custom changes inside the Docker daemon and created containerd."

✅ "You can imagine it's the same as Docker — everything is the same, the name is different, with some custom changes."

🔍 **The tidy version, since this trips people up:** `containerd` was originally *extracted out of Docker* — Docker itself uses it internally. Kubernetes used to reach Docker through a shim layer, and in 2022 it **removed that shim** and started talking to `containerd` directly. So it's less "Docker wasn't good enough" and more "Kubernetes cut out the middleman and kept the engine". Your images still work identically — they're the same standard image format either way.



15 📦 Pod, Node, Container — The Nesting

```
CLUSTER
├── NODE (a server)
│   ├── POD ← ★ the smallest unit in Kubernetes
│   │   ├── CONTAINER (one, or a few)
│   │   │   └── + VOLUME (for data that must survive)
```

🎯 INTERVIEW ALERT

She flagged it explicitly: "This is also a viva-round question: which is the smallest unit inside a Kubernetes cluster? A pod."

🧠 PICTURE THIS

Her explanation: "A pod is like one big container, one big environment, and inside that pod you have containers. One big box, inside that box your containers are running. And now we have a very big container where we are adding pods — that's the node."

How many containers in a pod?

✓ LOCK THIS IN

"Maybe one, maybe two, maybe three. In actual industry we are running only one container — one container, and one sidecar container for logging and other purposes."

🔍 Init container vs sidecar — these are NOT the same thing

She said: "One is your main container and second one is an init container. Init container means a sidecar."

🔍 CLARIFIED

Simplified, because these are genuinely two different things and mixing them up will cost you an interview:

	Init container	Sidecar container
When it runs	before the main container, then exits	alongside the main container, for the pod's whole life
Runs at the same time as the app?	✗ never	✓ always
Typical job	wait for the DB, run a migration, fetch config	ship logs, collect metrics, service-mesh proxy
Analogy	the crew who set up the stage and leave	the sound engineer who stays for the whole gig

She described the *sidecar* correctly — "*a sidecar container for log purposes*" is exactly right. It's just the name **init container** that got attached to it. In her course, the concept she's teaching is the **sidecar**. She said sidecars would be covered properly next lecture.

📁 Kubernetes volumes

A student asked what happens to a database container's data if the container dies.

✓ LOCK THIS IN

"If you're adding a volume, surely it will be there. If the database container is killed, crashed, deleted or stopped — it's going to create a new container, but what about my data? Data will be stored in the volume."

💡 The pattern repeats exactly:

Docker	Kubernetes
Docker volume	Kubernetes volume
Docker network	Kubernetes network policies



16 ♦ Career Asides

🕸 Service mesh

🎤 SAID IN CLASS

"Maybe you heard about service mesh — a very advanced topic in Kubernetes. If you know this thing now, I mean, any company will hire you. If you're doing some projects related to service mesh and the basic concepts, it's very very easy. They need a skilful person who knows the actual concept."

⚠️ **But she was honest about it:** "Service mesh is one of the headaches we faced in our company. It's very, very tough to understand — which cluster, which node, and then you're able to do port forwarding and all this stuff."

💡 She also noted the roles it creates: a **Kubernetes administrator** managing the cluster, and a **Kubernetes developer**.

🏆 The Kubernetes certifications

📌 NOTE

The five CNCF certifications: **CKA** (Administrator) · **CKAD** (Application Developer) · **CKS** (Security) · **KCNA** (Cloud Native Associate) · and one more.

✓ **Do all five and you become a "Kubestronaut."** 🚀

Cost	~\$300–500 each — "this is the costliest certification I've ever seen"
Attempts	two chances to clear it, within a year
Discounts	"you can get 50 or 60% discount"; a friend's community offered her up to 80% off

Reimbursement *"Then I have to reimburse from my company. And the company said 'I will not.' And then I said okay, then I will not do the certification." 😞*

🕒 **Her actual strategy, and it's good advice:**

"If you're getting it free, do it anyhow. Even if at midnight someone pings you that this certification is free — just do it. Because you know, after one hour, after two hours, maybe after one month, this certification will cost you around \$200."

The receipt: *"When I started Oracle certifications there were six certifications — Oracle Cloud Architect, Cloud Engineer, one Solution, one DevOps-related. We just cleared them overnight, in just a month. Those certifications cost around \$700+. I did it completely free. After one month they went from free to full price."*

✅ *"When you get a chance, don't think twice. If you're thinking, then the chance will go — surely."*



17 📝 Homework

Task

- 1 🌊 **Run the full 3-tier demo** from her repo ([session8/demo/](#)) with `docker compose up -d`
- 2 ✍️ **From the frontend container**, `docker exec` in and `curl http://backend:5000` — confirm you can reach it
- 3 ✍️ **Now try the database the same way** — *"you need to check if you're able to do the same thing with the database or not. You know the answer. I just need to check."* 😞
- 4 ⬇️ **Install Minikube** — minikube.sigs.k8s.io/docs/start
- 5 ▶️ Run `minikube start`, then `minikube status`
- 6 📖 **Read the Kubernetes architecture docs** — kubernetes.io/docs/concepts/architecture — click into **every** component: API server, etcd, scheduler, controller manager, kubelet, kube-proxy, container runtime
- 7 📖 Do the [Kubernetes Basics tutorial](#)
- 8 📄 Set up your **session README + Google Form** submission

💡 GOOD TO KNOW

Minikube in one line: *"Just do `minikube start`. It will start something like a container — just like whatever we do in Docker Compose, we can see the container is up. Same thing will happen."*

```
minikube start
minikube status
minikube stop
```

⚠ WATCH OUT

Next class's opening question: "Tomorrow we'll learn — if I'm running a command `kubectl get pods`, what actually happens in the background? But for that you need to know each component." That's why the reading matters.

📖 Her documentation advice

✅ LOCK THIS IN

"Always refer to the official website. If you're not able to understand, then and then start with GeeksforGeeks or exam-topic sites. I recommend starting with the Kubernetes documentation — you'll get more clarity, plus new updated content, because Kubernetes itself handles this documentation and it's up to date."

😄 A student suggested **W3Schools** ("the green colour website") — "Yeah, that is also a good website."



18



Cheat-Sheet

Docker Compose

Command	Does
<code>docker compose up</code>	start (foreground, logs visible)
<code>docker compose up -d</code>	start in background 🌟
<code>docker compose up -d --build</code>	rebuild images first
<code>docker compose ps</code>	list this project's containers
<code>docker compose down</code>	stop and remove everything

Compose file keys

Key	Does
<code>services:</code>	your containers (each entry = one service)

Key	Does
<code>image:</code>	pull a ready-made image
<code>build: ./folder</code>	build from a Dockerfile in that folder
<code>ports: - "8080:80"</code>	publish — host:container . Frontend only!
<code>environment:</code>	env variables (DB passwords, etc.)
<code>volumes:</code> (in a service)	mount a volume or bind mount
<code>networks:</code> (in a service)	which networks this service joins
<code>depends_on:</code>	start this service after the named one
<code>networks:</code> (top level)	declare networks — outside <code>services:</code>
<code>volumes:</code> (top level)	declare volumes — outside <code>services:</code>

Kubernetes vocabulary

Term	Means
Cluster	the whole environment — control plane + worker nodes
Control plane / master	the boss. Manages workloads, runs no applications
Worker node	a server where your containers actually run
Pod	★ the smallest unit in Kubernetes; holds one or more containers
Node	a server holding pods
etcd	key-value database holding the whole cluster's state
kube-apiserver	the front door — everything goes through it
kube-scheduler	decides which node a pod goes on
controller-manager	keeps actual state matching desired state
Node controller	are all my nodes alive?
ReplicaSet controller	are there exactly N pods?
kubelet	keeps containers on this node alive; sends the heartbeat
kube-proxy	networking / network policies
containerd	the CRI — the container runtime Kubernetes actually uses
Quorum	why you run 3 or 5 masters, not 2
Sidecar	a helper container running alongside your app (logging, metrics)
Init container	a container that runs before the app and exits

Term	Means
Service mesh	advanced networking layer — very hireable, very painful
Kubestronaut	someone who's cleared all five CNCF certs 🚀
kubectl	the CLI — "like <code>docker ps</code> but <code>kubectl get pods</code> "

kubectl commands mentioned

```
kubectl get pods
kubectl get nodes
kubectl get all
kubectl apply -f deployment.yaml
```

Minikube

```
minikube start
minikube status
minikube stop
```



19 Notes on the Transcript Itself

- **Instructor name:** the header credits "*Ritesh Prajapati*" — the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** ([@Nency-Ravaliya](#)).
- This transcript was generated by a **different classmate** than Classes 7–8 (`kabeer...` [@sst.scaler.com](#)), still using **Whisper large-v3-turbo**, so quality is good.
- 🔍 **Four places where a clarification was added rather than a silent correction**, each keeping her version first because that's what's testable: `depends_on` waits for *started*, *not ready* (§6) · **the kubelet↔controller loop goes through the API server** (§14) · **containerd was extracted from Docker rather than replacing it** (§14) · and most importantly **init container ≠ sidecar** (§15).
- Garbled terms corrected: "service smash" → **service mesh**, "kubastronaut" → **Kubestronaut**, "CK, CKD, CKS, KCNA" → **CKA, CKAD, CKS, KCNA**, "Duncan Composer / docker compost" → **Docker Compose**, "ATCD / etc DQ" → **etcd**, "quick proxy / QPROXY" → **kube-proxy**, "cube lid / kube lid" → **kubelet**, "Docker storm / Docker spam / Docker swap" → **Docker Swarm**, "key address" → **K8s**, "call" → `curl`, "wind mount" → **bind mount**, "redhead certification" → **Red Hat certification**, "duffer" → her joke about over-engineering, "Geek4Geeks" → **GeeksforGeeks**.

- 😊 A genuine **microphone failure** ate several minutes mid-class — *"Every instructor is out of the mic"* — and is not content.